

Cats vs. Dogs Image Classification using CNN — Detailed Summary Report

1. Project Overview

This project focuses on building an image classification system capable of distinguishing between images of cats and dogs. The solution is implemented using a Convolutional Neural Network (CNN) built with TensorFlow and Keras. The entire pipeline covers data acquisition, preprocessing, model design, training, evaluation, and prediction — making it a complete end-to-end deep learning project suitable for beginners and intermediate practitioners.

2. Objective

The primary goal is to train a CNN model that can:

- Accept an image as input
 - Automatically learn visual features such as edges, textures, and shapes
 - Output a prediction classifying the image as either a **Cat** or a **Dog** with a confidence score
-

3. Dataset

Source: Microsoft Cats and Dogs Dataset (publicly available) **Size:** Approximately 25,000 labeled images (~12,500 per class) **Classes:** Two — Cat and Dog **Format:** JPEG/PNG images of varying sizes and backgrounds

The dataset is downloaded programmatically using Python's urllib library and extracted using the zipfile module, making the process fully automated without any manual steps.

4. Data Preparation

4.1 Directory Structure

After downloading and extracting, the dataset is reorganized into the following structured format that TensorFlow's ImageDataGenerator expects:

```
dataset/  
├── train/  
│   ├── Cat/  
│   └── Dog/  
└── test/  
    ├── Cat/  
    └── Dog/
```

4.2 Train-Test Split

- **Training Set:** 80% of total images (~10,000 per class)
 - **Testing Set:** 20% of total images (~2,500 per class)
 - Images are **randomly shuffled** before splitting to ensure an unbiased distribution
 - Corrupt or unreadable image files are automatically detected and skipped during the copy process to prevent training errors
-

5. Data Augmentation and Preprocessing

5.1 Why Augmentation?

Real-world datasets are often limited. Augmentation artificially increases the diversity of the training set by applying random transformations to existing images. This teaches the model to recognize cats and dogs regardless of orientation, lighting, or position, significantly reducing overfitting.

5.2 Augmentation Techniques Applied (Training Set Only)

Technique	Value	Purpose
Rescaling	1/255	Normalize pixel values to [0, 1]
Rotation	$\pm 20^\circ$	Handle tilted images
Width Shift	20%	Handle horizontal displacement
Height Shift	20%	Handle vertical displacement
Shear	20%	Handle angular distortions
Zoom	20%	Handle close-up or distant images
Horizontal Flip	Yes	Mirror images for variety
Fill Mode	Nearest Fill empty pixels after transformation	

5.3 Test Set Preprocessing

The test set uses **only rescaling (normalization)** — no augmentation — so that evaluation reflects true real-world performance without artificial modifications.

5.4 Image Resizing

All images are resized to a uniform **150×150 pixels** to ensure consistent input dimensions for the CNN, as neural networks require fixed-size inputs.

6. Model Architecture

The CNN is built using Keras's Sequential API and consists of three main sections: **Convolutional Blocks**, a **Flatten Layer**, and a **Fully Connected Head**.

6.1 Full Architecture

Input Shape: (150, 150, 3) $\leftarrow$ Height $\times$ Width $\times$ RGB Channels

Block 1:

Conv2D $\rightarrow$ 32 filters, (3 $\times$ 3) kernel, ReLU activation

MaxPooling2D $\rightarrow$ (2 $\times$ 2) pool size

Block 2:

Conv2D $\rightarrow$ 64 filters, (3 $\times$ 3) kernel, ReLU activation

MaxPooling2D $\rightarrow$ (2 $\times$ 2) pool size

Block 3:

Conv2D $\rightarrow$ 128 filters, (3 $\times$ 3) kernel, ReLU activation

MaxPooling2D $\rightarrow$ (2 $\times$ 2) pool size

Flatten $\rightarrow$ Convert 3D feature maps to 1D vector

Dense $\rightarrow$ 128 neurons, ReLU activation

Dropout $\rightarrow$ 50% dropout rate (regularization)

Output Layer:

Dense $\rightarrow$ 2 neurons, Softmax activation $\leftarrow$ [Cat, Dog]

6.2 Layer-by-Layer Explanation

Convolutional Layers (Conv2D): These are the core of the CNN. Each filter slides across the image and learns to detect specific patterns — early layers learn simple features like edges and corners, while deeper layers learn complex features like ears, snouts, and fur textures. The number of filters increases ($32 \rightarrow 64 \rightarrow 128$) as the network goes deeper to capture increasingly complex patterns.

ReLU Activation: Rectified Linear Unit is applied after each convolution. It introduces non-linearity by converting all negative values to zero, allowing the network to learn complex, non-linear relationships in the data.

MaxPooling2D: After each convolution, a 2×2 max pooling operation reduces the spatial dimensions (height and width) by half, keeping only the most prominent feature in each region. This reduces computational cost and makes the model more robust to small shifts in the image.

Flatten: Converts the 3D output of the last pooling layer into a single 1D vector so it can be fed into standard dense (fully connected) layers.

Dense (128 neurons): A fully connected layer where every neuron connects to all inputs. This layer combines all the learned features to form a high-level understanding of the image.

Dropout (0.5): During training, 50% of the neurons in the previous dense layer are randomly turned off at each step. This prevents the model from memorizing the training data (overfitting) and forces it to learn more robust, generalizable patterns.

Output Layer — Dense (2 neurons, Softmax): The final layer produces two probability values that sum to 1.0 — one for Cat and one for Dog. The class with the higher probability is chosen as the final prediction.

7. Model Compilation

The model is compiled with the following configuration:

Component	Choice	Reason
Optimizer	SGD (lr=0.01, momentum=0.9)	Stochastic Gradient Descent with momentum for stable, reliable convergence
Loss Function	Categorical Cross-Entropy	Standard loss for multi-class classification with one-hot encoded labels
Metric	Accuracy	Easy-to-interpret performance measure

Why SGD with Momentum? Momentum helps SGD move in the right direction more consistently by accumulating past gradient information, preventing it from oscillating and helping it converge faster than plain SGD.

Why Categorical Cross-Entropy? Since labels are one-hot encoded (Cat = [1,0], Dog = [0,1]) and the output uses Softmax, categorical cross-entropy is the mathematically correct loss function. It penalizes confident wrong predictions heavily, pushing the model to improve.

8. Training

- **Epochs:** 15
- **Batch Size:** 32
- **Validation:** The test generator is used as the validation set during training, so performance on unseen data is monitored after every epoch
- **Steps per Epoch:** Automatically calculated based on total training images divided by batch size

During training, TensorFlow reports four values per epoch:

- `loss` — training loss
- `accuracy` — training accuracy
- `val_loss` — validation loss
- `val_accuracy` — validation accuracy

Monitoring `val_accuracy` and `val_loss` side by side with training metrics is critical for detecting overfitting — if training accuracy keeps rising but validation accuracy stagnates or drops, the model is overfitting.

9. Evaluation

After training completes, the model is evaluated on the full test set using `model.evaluate()`, which provides:

- **Final Test Loss** — measures how far predictions are from the truth
- **Final Test Accuracy** — percentage of correctly classified images

9.1 Training History Plot

A dual-panel matplotlib plot is automatically generated and saved as `training_history.png`:

- **Left Panel:** Training vs Validation Accuracy across all epochs
- **Right Panel:** Training vs Validation Loss across all epochs

These curves visually reveal whether the model is underfitting, overfitting, or generalizing well.

Ideal outcome: Both training and validation accuracy increase together and converge, while both losses decrease and converge — indicating good generalization.

10. Model Saving

After training, the model is saved in Keras format:

```
cats_dogs_model.keras
```

This allows the trained model to be reloaded at any time in a future session without retraining, using:

```
model = tf.keras.models.load_model("cats_dogs_model.keras")
```

11. Prediction System

Two prediction utilities are implemented:

11.1 Single Image Prediction — `predict_image()`

- Loads an image from a given file path
- Resizes it to 150×150 and normalizes pixel values
- Adds a batch dimension to match the model's expected input shape
- Passes it through the model and reads the softmax output
- Displays the image with the predicted label and confidence percentage

Example Output:

Prediction : Dog (94.3% confidence)

11.2 Batch Prediction — `predict_batch()`

- Scans an entire folder for image files
- Runs prediction on each image individually
- Prints a formatted table showing the filename, predicted class, and confidence for all images

Example Output:

Image	Prediction	Confidence

my_photo1.jpg	Cat	87.6%
my_photo2.jpg	Dog	95.1%
test_animal.png	Cat	78.3%

12. Key Design Decisions

Decision	Rationale
Image size 150×150	Balance between detail and computational cost
3 Conv blocks	Sufficient depth for a binary classification task without overcomplicating
Increasing filter sizes (32→64→128)	Capture increasingly abstract features at deeper layers
Dropout at 0.5	Standard regularization rate proven effective for mid-sized CNNs
SGD over Adam	More interpretable, stable for simple tasks; meets the project requirement
Categorical over Binary Cross-Entropy	Architecture uses 2-neuron softmax output, making categorical cross-entropy the correct pairing

13. Possible Improvements

- **Transfer Learning:** Use a pre-trained model like VGG16, ResNet50, or MobileNet as the base — this can push accuracy above 95% with far fewer epochs
- **Learning Rate Scheduling:** Gradually reduce the learning rate as training progresses to fine-tune the model in later epochs
- **Batch Normalization:** Add batch normalization layers after convolutions to speed up training and improve stability

- **More Epochs with Early Stopping:** Train for more epochs but use Keras's EarlyStopping callback to stop automatically when validation accuracy stops improving
 - **Class Activation Maps (CAM):** Visualize which regions of the image the model focuses on for its predictions — useful for interpretability
-

14. Summary

This project successfully demonstrates how to build a complete image classification pipeline from scratch using CNNs. Starting from raw data download all the way to making predictions on new images, every step is implemented in clean, beginner-friendly Python code. The model learns to classify cats and dogs by automatically extracting and combining visual features through convolutional layers, and regularization techniques like Dropout ensure the model generalizes well to images it has never seen before. The saved model can be deployed in any environment for real-world use.